

LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Insection dan Bubble sort”

disusun Oleh:

Muhammad Faiz An Anri

NIM 2511532021

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

KATA PENGANTAR

Ucapan syukur kita lafalkan kepada Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Merge sort bekerja dengan membagi array menjadi dua bagian secara rekursif hingga tersisa satu elemen, lalu menggabungkannya kembali dalam urutan yang benar — stabil dan konsisten di $O(n \log n)$ tapi butuh memori ekstra. Quick sort memilih sebuah pivot lalu mempartisi array sehingga elemen lebih kecil ada di kiri dan lebih besar di kanan, kemudian rekursif pada kedua sisi — rata-rata $O(n \log n)$ dan in-place, tapi bisa degradasi ke $O(n^2)$ jika pemilihan pivot buruk. Shell sort adalah pengembangan insertion sort yang menyortir elemen-elemen yang berjarak gap tertentu terlebih dahulu, lalu memperkecil gap secara bertahap hingga 1 — tidak seefisien dua yang lain secara teoritis, tapi sederhana, in-place, dan performanya cukup baik untuk data berukuran sedang.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Binary Tree
2. Memahami konsep dasar Graph Traversal
3. Memahami konsep dasar Graph

BAB 2 PEMBAHASAN

3.1 Program Binary Tree

```

2  package Pekan9_2511532021;
3
4  public class BTree_2511532021 {
5      private Node_2511532021 root_2021;
6      private Node_2511532021 currentNode_2021;
7      public BTree_2511532021() {
8          root_2021 = null;
9      }
10
11     public boolean search_2021(int data_2021) {
12         return search_2021(root_2021, data_2021);
13     }
14
15     private boolean search_2021(Node_2511532021 node_2021, int data_2021) {
16         if (node_2021.getData_2021() == data_2021) {
17             return true;
18         }
19         if (node_2021.getLeft_2021() != null) {
20             if (search_2021(node_2021.getLeft_2021(), data_2021)) {
21                 return true;
22             }
23         }
24         if (node_2021.getRight_2021() != null) {
25             if (search_2021(node_2021.getRight_2021(), data_2021)) {
26                 return true;
27             }
28         }
29         return false;
30     }
31
32     public void printInOrder_2021() {
33         root_2021.printInOrder_2021(root_2021);
34     }
35
36     public void printPreOrder_2021() {
37         root_2021.printPreorder_2021(root_2021);
38     }
39
40     public void printPostOrder() {
41         root_2021.printPostOrder_2021(root_2021);
42     }
43
44     public Node_2511532021 getRoot_2021() {
45         return root_2021;
46     }
47
48     public boolean isEmpty_2021() {
49         return root_2021 == null;
50     }
51
52     public int countNode_2021() {
53         return countNode_2021(root_2021);
54     }
55
56     private int countNode_2021(Node_2511532021 node_2021) {
57         int count_2021 = 1;
58         if (node_2021 == null) {
59             return 0;
60         }
61         else {
62             count_2021 += countNode_2021(node_2021.getLeft_2021());
63             count_2021 += countNode_2021(node_2021.getRight_2021());
64             return count_2021;
65         }
66     }
67

```

```

68     }
69
70     public void print_2021() {
71         root_2021.print_2021();
72     }
73
74     public Node_2511532021 getCurrent_2021() {
75         return currentNode_2021;
76     }
77
78     public void setCurrent_2021(Node_2511532021 node_2021) {
79         this.currentNode_2021 = node_2021;
80     }
81
82     public void setRoot_2021(Node_2511532021 node_2021) {
83         this.root_2021 = node_2021;
84     }
85 }

```

Kode ini mendefinisikan kelas BTree_2511532021, sebuah implementasi struktur data binary tree dengan dua atribut utama: root_2021 (akar pohon) dan currentNode_2021 (penanda node yang sedang aktif/dikerjakan). Method search_2021 mencari sebuah nilai dalam pohon secara rekursif dengan pola traversal: cek node saat ini, lalu cari di subtree kiri, baru subtree kanan, mengembalikan true begitu nilai ditemukan di salah satu cabang. Ada tiga method print (printlnOrder_2021, printPreOrder_2021, printPostOrder) yang masing-masing memanggil method traversal sejenis pada objek Node_2511532021, jadi logika pencetakannya sebenarnya didelegasikan ke kelas Node, bukan diimplementasikan di sini.

Method countNode_2021 menghitung total jumlah node dengan rekursi: untuk setiap node non-null ditambahkan 1, lalu dijumlahkan dengan hasil rekursi pada anak kiri dan kanan — base case-nya mengembalikan 0 saat node null. Sisanya adalah method-method pelengkap (getter/setter dan utilitas) seperti isEmpty_2021 (cek apakah root null), getRoot_2021/setRoot_2021, serta getCurrent_2021/setCurrent_2021 untuk mengelola currentNode_2021.

1.1 Program Node

```

86 package Pekan9_2511532021;
87
88 public class Node_2511532021 {
89     int data_2021;
90     Node_2511532021 left_2021;
91     Node_2511532021 right_2021;
92     public Node_2511532021 (int data_2021) {
93         this.data_2021 = data_2021;
94         left_2021 = null;
95         right_2021 = null;
96     }
97
98     public void setLeft_2021(Node_2511532021 node_2021) {

```

```

99         if (left_2021 == null) {
100             left_2021 = node_2021;
101         }
102     }
103
104     public void setRight_2021(Node_2511532021 node_2021) {
105         if (right_2021 == null) {
106             right_2021 = node_2021;
107         }
108     }
109
110     public Node_2511532021 getLeft_2021() {
111         return left_2021;
112     }
113     public Node_2511532021 getRight_2021() {
114         return right_2021;
115     }
116
117     public void setData_2021(int data_2021) {
118         this.data_2021 = data_2021;
119     }
120
121     public int getData_2021() {
122         return this.data_2021;
123     }
124
125     void printPreorder_2021(Node_2511532021 node_2021) {
126         if (node_2021 == null) {
127             return ;
128         }
129
130         System.out.print(node_2021.data_2021 + " ");
131         printPreorder_2021(node_2021.left_2021);
132         printPreorder_2021(node_2021.right_2021);
133     }
134
135     void printPostOrder_2021(Node_2511532021 node_2021) {
136         if (node_2021 == null) {
137             return;
138         }
139         printPostOrder_2021(node_2021.left_2021);
140         printPostOrder_2021(node_2021.right_2021);
141         System.out.print(node_2021.data_2021 + " ");
142     }
143
144     void printInOrder_2021(Node_2511532021 node_2021) {
145         if( node_2021 == null) {
146             return ;
147         }
148
149         printInOrder_2021(node_2021.left_2021);
150         System.out.print(node_2021.data_2021+ " ");
151         printInOrder_2021(node_2021.right_2021);
152     }
153
154     public String print_2021() {
155         return this.print_2021("", true, "");
156     }
157
158
159     public String print_2021(String prefix_2021, boolean isTail_2021, String sb_2021) {
160         if (right_2021 != null) {
161             right_2021.print_2021(prefix_2021 +( isTail_2021 ? "|" : "
162 "),false, sb_2021);
163         }
164         System.out.println(prefix_2021+(isTail_2021 ? "\\--" : "/--")+data_2021);
165         if (left_2021 != null) {
166             left_2021.print_2021(prefix_2021 +(isTail_2021 ? "
167 "),true, sb_2021);
168         }
169         return sb_2021;
170     }

```

Kelas `Node_2511532021` ini merepresentasikan sebuah node dalam binary tree, dengan tiga atribut: `data_2021` (nilai integer yang disimpan), serta `left_2021` dan `right_2021` (referensi ke anak kiri dan kanan). Constructor menerima sebuah nilai data dan menginisialisasi kedua child sebagai null. Method `setLeft_2021` dan `setRight_2021` memiliki perilaku khusus: keduanya hanya akan mengisi child jika posisi tersebut masih null — jadi tidak bisa menimpa (overwrite) child yang sudah ada, hanya bisa diisi sekali. Selain itu ada method getter/setter standar untuk data dan kedua child.

Bagian kedua berisi tiga method traversal rekursif yang dipanggil dari kelas `BTree_2511532021`: `printPreorder_2021` mencetak node terlebih dahulu sebelum rekursi ke kiri lalu kanan (root-left-right), `printInOrder_2021` mencetak setelah rekursi kiri tapi sebelum kanan (left-root-right), dan `printPostOrder_2021` mencetak setelah kedua anak diproses (left-right-root) — ketiganya menggunakan base case if (node == null) return. Terakhir, ada method `print_2021` yang melakukan visualisasi pohon secara vertikal/grafis ke console menggunakan indentasi dan karakter `--/--` untuk menunjukkan struktur cabang; menariknya, parameter `sb_2021` (`StringBuilder`) sebenarnya tidak pernah digunakan untuk membangun string — method ini langsung mencetak dengan `System.out.println` dan mengembalikan `sb_2021` yang kosong, jadi return value-nya pada praktiknya tidak berguna (dead code).

1.1 Program Graph Traversal

```

171 package Pekan9_2511532021;
172 import java.util.*;
173
174 public class GraphTraversal_2511532021 {
175     private Map<String, List<String>> graph_2021 = new HashMap<>();
176
177     public void addEdge_2021(String node1_2021, String node2_2021) {
178         graph_2021.putIfAbsent(node1_2021, new ArrayList<>());
179         graph_2021.putIfAbsent(node2_2021, new ArrayList<>());
180         graph_2021.get(node1_2021).add(node2_2021);
181         graph_2021.get(node2_2021).add(node1_2021);
182     }
183
184     public void printGraph_2021() {
185         System.out.println("Graf Awal (Adjacency List): ");
186         for (String node_2021 : graph_2021.keySet()) {
187             List<String> neighbors_2021 = graph_2021.get(node_2021);
188             System.out.println(node_2021 + " -> " + String.join(", ",
neighbors_2021));
189         }
190     }
191
192     public void dfs_2021(String start_2021) {
193         Set<String> visited_2021 = new HashSet<>();
194         System.out.println("Penelusuran DFS");

```



```

195         dfsHelper_2021(start_2021, visited_2021);
196         System.out.println();
197     }
198
199     private void dfsHelper_2021(String current_2021, Set<String> visited_2021) {
200         if (visited_2021.contains(current_2021)) {
201             return;
202         }
203         visited_2021.add(current_2021);
204         System.out.print(current_2021 + " ");
205         for (String neighbor_2021 : graph_2021.getOrDefault(current_2021, new
ArrayList<>())) {
206             dfsHelper_2021(neighbor_2021, visited_2021);
207         }
208     }
209
210     public void bfs_2021(String start) {
211         Set<String> visited_2021 = new HashSet<>();
212         Queue<String> queue_2021 = new LinkedList<>();
213         queue_2021.add(start);
214         visited_2021.add(start);
215         System.out.println("Penelusuran BFS:");
216         while (!queue_2021.isEmpty()) {
217             String current = queue_2021.poll();
218             System.out.print(current + " ");
219             for (String neighbor : graph_2021.getOrDefault(current, new ArrayList<>())) {
220                 if (!visited_2021.contains(neighbor)) {
221                     queue_2021.add(neighbor);
222                     visited_2021.add(neighbor);
223                 }
224             }
225         }
226         System.out.println();
227     }
228
229     public static void main(String[] args) {
230         GraphTraversal_2511532021 graph_2021 = new GraphTraversal_2511532021();
231
232         graph_2021.addEdge_2021("A", "B");
233         graph_2021.addEdge_2021("A", "C");
234         graph_2021.addEdge_2021("B", "D");
235         graph_2021.addEdge_2021("B", "E");
236         System.out.println("Graf Awal adalah: ");
237         graph_2021.printGraph_2021();
238         graph_2021.dfs_2021("A");
239         graph_2021.bfs_2021("A");
240     }
241 }
242

```

Program ini mengimplementasikan Quick Sort dengan median-of-three pivot. `medianOfThree_2021()` memilih pivot terbaik dari tiga elemen (low, mid, high) dengan mengurutkannya lalu menempatkan median di posisi high sebagai pivot. `partition_2021()` memanggil median-of-three lalu mempartisi array — elemen lebih kecil dari pivot dikumpulkan ke kiri, lebih besar ke kanan, lalu pivot ditempatkan di posisi akhirnya dan indexnya dikembalikan. `quickSort_2021()` secara rekursif memanggil dirinya pada sisi kiri dan kanan pivot hingga sub-array berukuran 1 — lalu di `main()` array {10,7,8,9,1,5} diurutkan dan dicetak sebelum dan sesudah sorting.

Kelas `GraphTraversal_2511532021` merepresentasikan graf tak berarah menggunakan adjacency list berbasis `HashMap<String, List<String>>`, di mana

`addEdge_2021` menambahkan edge dua arah (kedua node saling menjadi tetangga satu sama lain), `printGraph_2021` mencetak seluruh daftar tetangga setiap node, sementara `dfs_2021` melakukan penelusuran depth-first secara rekursif (mengunjungi satu node, menandainya visited, lalu rekursif ke setiap tetangga yang belum dikunjungi) dan `bfs_2021` melakukan penelusuran breadth-first menggunakan Queue (memproses node secara berlapis dengan menambahkan tetangga yang belum dikunjungi ke antrian); pada main, dibuat graf dengan edge A-B, A-C, B-D, B-E lalu dicetak adjacency list-nya, dan dijalankan DFS serta BFS dari node "A" — untuk graf ini hasil DFS akan mencetak A B D E C (karena dari A ke B dulu, lalu eksplorasi habis dari B sebelum balik ke C) dan BFS akan mencetak A B C D E (karena A diproses dulu sehingga semua tetangga langsungnya B dan C masuk antrian sebelum D dan E).

1.1 Program BTreeDriver

```

243 package Pekan9_2511532021;
244
245 public class BTreeDriver_2511532021 {
246     public static void main(String[] args) {
247         BTree_2511532021 tree_2021 = new BTree_2511532021();
248         System.out.print("Jumlah simpul awal pohon : ");
249         System.out.println(tree_2021.countNode_2021());
250
251         Node_2511532021 root_2021 = new Node_2511532021(1);
252
253         tree_2021.setRoot_2021(root_2021);
254
255         System.out.println("Jumlah simpul jika hanya ada root ");
256         System.out.println(tree_2021.countNode_2021());
257
258         Node_2511532021 Node2_2021 = new Node_2511532021(2);
259         Node_2511532021 Node3_2021 = new Node_2511532021(3);
260         Node_2511532021 Node4_2021 = new Node_2511532021(4);
261         Node_2511532021 Node5_2021 = new Node_2511532021(5);
262         Node_2511532021 Node6_2021 = new Node_2511532021(6);
263         Node_2511532021 Node7_2021 = new Node_2511532021(7);
264         Node_2511532021 Node8_2021 = new Node_2511532021(8);
265         Node_2511532021 Node0_2021 = new Node_2511532021(9);
266         root_2021.setLeft_2021(Node2_2021);
267         Node2_2021.setLeft_2021(Node4_2021);
268         Node2_2021.setRight_2021(Node5_2021);
269         Node4_2021.setRight_2021(Node8_2021);
270         root_2021.setRight_2021(Node3_2021);
271         Node3_2021.setLeft_2021(Node6_2021);
272         Node3_2021.setRight_2021(Node7_2021);
273         Node6_2021.setLeft_2021(Node0_2021);
274
275         tree_2021.setCurrent_2021(tree_2021.getRoot_2021());
276         System.out.println("menampilkan simpul terakhir: ");
277         System.out.println(tree_2021.getCurrent_2021().getData_2021());
278         System.out.println("Jumlah simpul ; setelah simpul 7 ditambahkan");
279         System.out.println(tree_2021.countNode_2021());
280         System.out.println("inOrder");
281         tree_2021.printInOrder_2021();
282         System.out.println("\nPreOrder");
283         tree_2021.printPreOrder_2021();
284         System.out.println("\nPostOrder");
285         tree_2021.printPostOrder();
286
287         System.out.println("\nMenampilkan simpul dalam bentuk pohon");
288         tree_2021.print_2021();
289     }
290 }
291

```

Driver class ini menguji fungsionalitas BTree_2511532021 dan Node_2511532021 dengan membangun pohon biner secara manual lalu menjalankan berbagai operasi. Pertama, dibuat tree kosong dan dicek countNode_2021() yang akan mengembalikan 0 (karena root_2021 masih null dan base case rekursi menangani ini dengan benar). Kemudian root diset dengan node bernilai 1, dan struktur pohon dibangun secara manual menggunakan setLeft_2021/setRight_2021 membentuk pohon dengan 9 node total: root(1) memiliki left=2, right=3; node 2 memiliki left=4, right=5; node 4 memiliki right=8; node 3 memiliki left=6, right=7;

dan node 6 memiliki left=9.

Setelah itu, `setCurrent_2021` diset ke root dan dicetak datanya (akan menampilkan 1, bukan "simpul terakhir" sebenarnya — label print-nya agak menyesatkan karena yang ditampilkan adalah root, bukan node terakhir yang ditambahkan). Lalu `countNode_2021()` dipanggil lagi dan akan menghasilkan 9 (sesuai jumlah node yang dibuat). Tiga traversal dijalankan: `inOrder` akan mencetak 4 8 2 5 1 9 6 3 7, `preOrder` akan mencetak 1 2 4 8 5 3 6 9 7, dan `postOrder` akan mencetak 8 4 5 2 9 6 7 3 1. Terakhir, `print_2021()` menampilkan visualisasi pohon secara vertikal ke console menggunakan format indentasi dengan karakter `--` dan `/--` untuk menggambarkan hierarki cabang kiri-kanan.

BAB 3 TUGAS

1.1 Program PetaKampus

```
292 package Pekan9_2511532021;
293
294 import javax.swing.*;
295 import java.awt.*;
296 import java.util.*;
297 import java.util.List;
298
299 public class PetaKampus_2511532021 extends JFrame {
300
301     private Map<String, List<String>> graph_2021 = new LinkedHashMap<>();
302     private Map<String, Point> posisi_2021 = new LinkedHashMap<>();
303
304     private List<String> jalur_2021 = new ArrayList<>();
305     private Set<String> dikunjungi_2021 = new LinkedHashSet<>();
306     private String startNode_2021 = null;
307     private String goalNode_2021 = null;
308
309     private GraphPanel_2021 panelGraph_2021;
310     private JComboBox<String> comboAwal_2021;
311     private JComboBox<String> comboTujuan_2021;
312     private JTextArea areaHasil_2021;
313     private JButton btnBFS_2021, btnDFS_2021, btnReset_2021;
314
315     public PetaKampus_2511532021() {
316         super("PENCARIAN JALUR - PETA KAMPUS UNAND (2511532021)");
317         buatGraph_2021();
318         buatGUI_2021();
319         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
320         setSize(950, 650);
321         setLocationRelativeTo(null);
322     }
323
324     private void buatGraph_2021() {
325         posisi_2021.put("Gerbang", new Point(60, 280));
326         posisi_2021.put("Rektorat", new Point(230, 150));
327         posisi_2021.put("Perpustakaan", new Point(230, 400));
328         posisi_2021.put("FTI", new Point(420, 400));
329         posisi_2021.put("FE", new Point(420, 100));
330         posisi_2021.put("FH", new Point(420, 480));
331         posisi_2021.put("PKM", new Point(580, 150));
332         posisi_2021.put("Masjid", new Point(580, 280));
333         posisi_2021.put("GOR", new Point(760, 100));
334         posisi_2021.put("Asrama", new Point(760, 280));
335         posisi_2021.put("Kantin", new Point(600, 430));
336         posisi_2021.put("LabKomputer", new Point(760, 430));
337
338         for (String node_2021 : posisi_2021.keySet()) {
339             graph_2021.put(node_2021, new ArrayList<>());
340         }
341
342         tambahEdge_2021("Gerbang", "Rektorat");
343         tambahEdge_2021("Gerbang", "Perpustakaan");
344         tambahEdge_2021("Rektorat", "PKM");
345         tambahEdge_2021("Rektorat", "FE");
346         tambahEdge_2021("Perpustakaan", "FTI");
347         tambahEdge_2021("Perpustakaan", "FH");
348         tambahEdge_2021("FTI", "LabKomputer");
349         tambahEdge_2021("FTI", "Kantin");
350         tambahEdge_2021("FE", "Kantin");
351         tambahEdge_2021("FE", "PKM");
352         tambahEdge_2021("FH", "Masjid");
353         tambahEdge_2021("PKM", "Masjid");
354         tambahEdge_2021("PKM", "GOR");
355         tambahEdge_2021("GOR", "Asrama");
356         tambahEdge_2021("Asrama", "Kantin");
357         tambahEdge_2021("Masjid", "Asrama");
358         tambahEdge_2021("LabKomputer", "PKM");
359     }
360
361     private void tambahEdge_2021(String a_2021, String b_2021) {
```

```

362         graph_2021.get(a_2021).add(b_2021);
363         graph_2021.get(b_2021).add(a_2021);
364     }
365
366     private void BFS_2021(String start_2021, String goal_2021) {
367         dikunjungi_2021.clear();
368         jalur_2021.clear();
369
370         Map<String, String> induk_2021 = new HashMap<>();
371         Queue<String> queue_2021 = new LinkedList<>();
372
373         queue_2021.add(start_2021);
374         dikunjungi_2021.add(start_2021);
375
376         boolean ditemukan_2021 = false;
377
378         while (!queue_2021.isEmpty()) {
379             String current_2021 = queue_2021.poll();
380
381             if (current_2021.equals(goal_2021)) {
382                 ditemukan_2021 = true;
383                 break;
384             }
385
386             for (String tetangga_2021 : graph_2021.get(current_2021)) {
387                 if (!dikunjungi_2021.contains(tetangga_2021)) {
388                     dikunjungi_2021.add(tetangga_2021);
389                     induk_2021.put(tetangga_2021, current_2021);
390                     queue_2021.add(tetangga_2021);
391                 }
392             }
393         }
394
395         if (ditemukan_2021) {
396             String node_2021 = goal_2021;
397             while (node_2021 != null) {
398                 jalur_2021.add(0, node_2021);
399                 node_2021 = induk_2021.get(node_2021);
400             }
401         }
402     }
403
404     private void DFS_2021(String start_2021, String goal_2021) {
405         dikunjungi_2021.clear();
406         jalur_2021.clear();
407         dfsHelper_2021(start_2021, goal_2021);
408     }
409
410     private boolean dfsHelper_2021(String current_2021, String goal_2021) {
411         dikunjungi_2021.add(current_2021);
412         jalur_2021.add(current_2021);
413
414         if (current_2021.equals(goal_2021)) {
415             return true;
416         }
417
418         for (String tetangga_2021 : graph_2021.get(current_2021)) {
419             if (!dikunjungi_2021.contains(tetangga_2021)) {
420                 if (dfsHelper_2021(tetangga_2021, goal_2021)) {
421                     return true;
422                 }
423             }
424         }
425
426         jalur_2021.remove(jalur_2021.size() - 1);
427         return false;
428     }
429
430     private void displayPath_2021(String namaAlgoritma_2021) {
431         StringBuilder sb_2021 = new StringBuilder();
432         sb_2021.append("Hasil Pencarian (" + namaAlgoritma_2021 + ") \n");
433         sb_2021.append("=====\n");
434
435         if (jalur_2021.isEmpty()) {

```

```

436         sb_2021.append("Jalur tidak ditemukan dari ")
437         .append(startNode_2021).append(" ke
438     ")}.append(goalNode_2021).append("\n");
439     } else {
440         sb_2021.append("Jalur : ").append(String.join(" -> ",
441     jalur_2021)).append("\n");
442     }
443     sb_2021.append("Node Dikunjungi : ")
444     .append(String.join(", ", dikunjungi_2021)).append("\n");
445     sb_2021.append("Jumlah Node Dieksplorasi :
446     ").append(dikunjungi_2021.size()).append("\n");
447     areaHasil_2021.setText(sb_2021.toString());
448     panelGraph_2021.repaint();
449 }
450 private void resetGraph_2021() {
451     jalur_2021.clear();
452     dikunjungi_2021.clear();
453     startNode_2021 = null;
454     goalNode_2021 = null;
455     areaHasil_2021.setText("Hasil Pencarian : \nJalur : \nNode Dikunjungi
456     : \nJumlah Node Dieksplorasi : 0");
457     panelGraph_2021.repaint();
458 }
459 private void buatGUI_2021() {
460     setLayout(new BorderLayout(10, 10));
461
462     JPanel panelAtas_2021 = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 10));
463     panelAtas_2021.setBorder(BorderFactory.createTitledBorder("Kontrol Pencarian"));
464
465     Vector<String> daftarNode_2021 = new Vector<>(graph_2021.keySet());
466     comboAwal_2021 = new JComboBox<>(daftarNode_2021);
467     comboTujuan_2021 = new JComboBox<>(daftarNode_2021);
468     comboTujuan_2021.setSelectedIndex(daftarNode_2021.size() - 1);
469
470     btnBFS_2021 = new JButton("BFS");
471     btnDFS_2021 = new JButton("DFS");
472     btnReset_2021 = new JButton("RESET");
473
474     btnBFS_2021.setBackground(new Color(120, 220, 120));
475     btnDFS_2021.setBackground(new Color(250, 200, 100));
476     btnReset_2021.setBackground(new Color(240, 120, 120));
477
478     panelAtas_2021.add(new JLabel("Lokasi Awal :"));
479     panelAtas_2021.add(comboAwal_2021);
480     panelAtas_2021.add(new JLabel("Lokasi Tujuan :"));
481     panelAtas_2021.add(comboTujuan_2021);
482     panelAtas_2021.add(btnBFS_2021);
483     panelAtas_2021.add(btnDFS_2021);
484     panelAtas_2021.add(btnReset_2021);
485
486     panelGraph_2021 = new GraphPanel_2021();
487     panelGraph_2021.setBorder(BorderFactory.createTitledBorder("Visualisasi Graph - Peta
488     Kampus Unand"));
489
490     areaHasil_2021 = new JTextArea(6, 50);
491     areaHasil_2021.setEditable(false);
492     areaHasil_2021.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 13));
493     resetGraph_2021();
494
495     JScrollPane scrollHasil_2021 = new JScrollPane(areaHasil_2021);
496     scrollHasil_2021.setBorder(BorderFactory.createTitledBorder("Hasil Pencarian"));
497
498     add(panelAtas_2021, BorderLayout.NORTH);
499     add(panelGraph_2021, BorderLayout.CENTER);
500     add(scrollHasil_2021, BorderLayout.SOUTH);
501
502     btnBFS_2021.addActionListener(e -> {
503         startNode_2021 = (String) comboAwal_2021.getSelectedItem();
504         goalNode_2021 = (String) comboTujuan_2021.getSelectedItem();
505         BFS_2021(startNode_2021, goalNode_2021);

```

```

505         displayPath_2021("BFS - Breadth First Search");
506     });
507
508     btnDFS_2021.addActionListener(e -> {
509         startNode_2021 = (String) comboAwal_2021.getSelectedItemAt();
510         goalNode_2021 = (String) comboTujuan_2021.getSelectedItemAt();
511         DFS_2021(startNode_2021, goalNode_2021);
512         displayPath_2021("DFS - Depth First Search");
513     });
514
515     btnReset_2021.addActionListener(e -> resetGraph_2021());
516 }
517
518 public void displayGraph_2021() {
519     panelGraph_2021.repaint();
520 }
521
522 private class GraphPanel_2021 extends JPanel {
523
524     private static final int RADIUS_2021 = 22;
525
526     public GraphPanel_2021() {
527         setBackground(Color.WHITE);
528     }
529
530     @Override
531     protected void paintComponent(Graphics g_2021) {
532         super.paintComponent(g_2021);
533         Graphics2D g2_2021 = (Graphics2D) g_2021;
534         g2_2021.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
535
536         g2_2021.setStroke(new BasicStroke(2));
537         for (String node_2021 : graph_2021.keySet()) {
538             Point p1_2021 = posisi_2021.get(node_2021);
539
540             for (String tetangga_2021 : graph_2021.get(node_2021)) {
541                 Point p2_2021 = posisi_2021.get(tetangga_2021);
542
543                 boolean adaDiJalur_2021 =
544                     jalur_2021.contains(node_2021)
545                     && jalur_2021.contains(tetangga_2021)
546                     && Math.abs(jalur_2021.indexOf(node_2021)
547                     - jalur_2021.indexOf(tetangga_2021)) == 1;
548
549                 if (adaDiJalur_2021) {
550                     g2_2021.setColor(Color.RED);
551                     g2_2021.setStroke(new BasicStroke(3));
552                 } else {
553                     g2_2021.setColor(Color.GRAY);
554                     g2_2021.setStroke(new BasicStroke(1));
555                 }
556
557                 g2_2021.drawLine(
558                     p1_2021.x, p1_2021.y,
559                     p2_2021.x, p2_2021.y
560                 );
561             }
562         }
563
564         for (Map.Entry<String, Point> entry_2021 : posisi_2021.entrySet()) {
565             String nama_2021 = entry_2021.getKey();
566             Point p_2021 = entry_2021.getValue();
567
568             Color warna_2021 = new Color(225, 225, 225);
569
570             if (dikunjungi_2021.contains(nama_2021))
571                 warna_2021 = new Color(255, 190, 80);
572
573             if (jalur_2021.contains(nama_2021))
574                 warna_2021 = new Color(255, 110, 110);
575
576             if (nama_2021.equals(startNode_2021))
577                 warna_2021 = new Color(110, 220, 110);

```



```

578
579         if (nama_2021.equals(goalNode_2021)
580             && !jalur_2021.contains(nama_2021))
581             warna_2021 = new Color(120, 160, 255);
582
583         if (nama_2021.equals(goalNode_2021)
584             && jalur_2021.contains(nama_2021))
585             warna_2021 = new Color(180, 80, 220);
586
587         g2_2021.setColor(warna_2021);
588         g2_2021.fillOval(
589             p_2021.x - RADIUS_2021,
590             p_2021.y - RADIUS_2021,
591             RADIUS_2021 * 2,
592             RADIUS_2021 * 2
593         );
594
595         g2_2021.setColor(Color.BLACK);
596         g2_2021.drawOval(
597             p_2021.x - RADIUS_2021,
598             p_2021.y - RADIUS_2021,
599             RADIUS_2021 * 2,
600             RADIUS_2021 * 2
601         );
602
603         FontMetrics fm_2021 = g2_2021.getFontMetrics();
604         int lebar_2021 = fm_2021.stringWidth(nama_2021);
605
606         g2_2021.drawString(
607             nama_2021,
608             p_2021.x - lebar_2021 / 2,
609             p_2021.y - RADIUS_2021 - 5
610         );
611     }
612 }
613
614 @Override
615 public Dimension getPreferredSize() {
616     return new Dimension(880, 540);
617 }
618 }
619
620 public static void main(String[] args) {
621     SwingUtilities.invokeLater(() -> {
622         PetaKampus_2511532021 frame_2021 = new PetaKampus_2511532021();
623         frame_2021.setVisible(true);
624         frame_2021.displayGraph_2021();
625     });
626 }
627 }

```

Program PetaKampus_2511532021 merupakan aplikasi berbasis Java Swing yang digunakan untuk mensimulasikan pencarian jalur antar lokasi di lingkungan kampus. Program memodelkan peta kampus sebagai sebuah graph, yaitu struktur data yang terdiri dari simpul (node) dan sisi (edge). Setiap node merepresentasikan lokasi seperti Gerbang, Rektorat, Perpustakaan, FTI, dan lainnya, sedangkan edge menunjukkan hubungan atau jalan yang menghubungkan dua lokasi. Struktur graph disimpan menggunakan *adjacency list* dengan Map<String, List<String>>, sehingga setiap lokasi memiliki daftar lokasi tetangga yang terhubung langsung dengannya.

Metode buatGraph_2021() bertugas membangun graph yang digunakan dalam program. Pada metode ini ditentukan posisi setiap node menggunakan objek Point

untuk keperluan visualisasi, kemudian dibuat hubungan antar node melalui metode `tambahEdge_2021()`. Karena hubungan dibuat dua arah, setiap kali sebuah edge ditambahkan maka kedua node akan saling dimasukkan ke daftar tetangga masing-masing. Dengan pendekatan ini, graph dapat menggambarkan jaringan jalan atau akses antar lokasi kampus secara sederhana namun efektif.

Algoritma Breadth First Search (BFS) diimplementasikan pada metode `BFS_2021()`. BFS bekerja dengan prinsip menjelajahi node berdasarkan tingkat kedekatan dari titik awal menggunakan struktur data Queue (antrian). Node yang ditemukan pertama kali akan diproses lebih dahulu sebelum berpindah ke node pada tingkat berikutnya. Dalam program ini BFS menyimpan informasi parent atau induk setiap node yang dikunjungi sehingga setelah tujuan ditemukan, jalur terpendek dapat direkonstruksi dengan menelusuri parent dari node tujuan hingga kembali ke node awal. Oleh karena itu BFS sangat cocok digunakan untuk mencari jalur dengan jumlah langkah paling sedikit pada graph yang tidak memiliki bobot.

Algoritma Depth First Search (DFS) diimplementasikan melalui metode `DFS_2021()` dan `dfsHelper_2021()`. Berbeda dengan BFS, DFS menggunakan pendekatan menyelam sedalam mungkin pada satu cabang sebelum kembali (*backtracking*) dan mencoba cabang lainnya. Implementasi ini menggunakan rekursi untuk mengunjungi node-node yang belum pernah dikunjungi. Setiap node yang dikunjungi dimasukkan ke dalam daftar jalur, dan apabila jalur tersebut tidak mengarah ke tujuan maka node terakhir akan dihapus melalui proses backtracking. DFS mampu menemukan jalur menuju tujuan, tetapi jalur yang diperoleh belum tentu merupakan jalur terpendek karena pencarian sangat dipengaruhi oleh urutan tetangga yang diperiksa.

Bagian antarmuka pengguna dibangun menggunakan komponen Swing seperti `JFrame`, `JPanel`, `JButton`, `JComboBox`, dan `JTextArea`. Pengguna dapat memilih lokasi awal dan tujuan, kemudian menjalankan BFS atau DFS melalui tombol yang tersedia. Hasil pencarian berupa jalur yang ditemukan, daftar node yang dikunjungi, dan jumlah node yang dieksplorasi akan ditampilkan pada area teks. Selain itu, kelas `GraphPanel_2021` bertugas menggambar graph secara visual, termasuk memberi warna berbeda pada node awal, node tujuan, node yang dikunjungi, serta jalur hasil pencarian. Dengan demikian program tidak hanya menunjukkan hasil algoritma

secara tekstual, tetapi juga memberikan visualisasi yang memudahkan pengguna memahami cara kerja BFS dan DFS pada graph.

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Terdapat 4 program yang dibuat pada praktikum kali ini, yaitu node, graph traversal, binary tree, dan binary tree driver.

Node adalah obyek utama yang digunakan pada keempat program. Berhubung seluruh materi berporos pada struktur data graf. Node memiliki beberapa interface dan memiliki beberapa atribut seperti left, right, dan data.

Lalu ada binary tree yang merupakan salah satu turunan dari struktur data graf. Memiliki beberapa method yang tertaut padanya, seperti print in order, post order, dan pre order.

Binary tree memiliki binary tree driver class sebagai method yang akan mereferensi pada interface binary tree

Terakhir ada graph traversal. Meliputi DFS dan BFS, depth first search menggunakan struktur data stack dan breadth first search menggunakan struktur data queue. Kedua traversal tersebut bertindak seluruhnya berbeda.

